Spring AI Roadmap

Basic to Advanced to Expert Learning Guide

This guide is designed as a structured study roadmap for developers who want to build, test, and operate AI-powered applications using the Spring ecosystem. It moves from prerequisites and first principles into production architecture, retrieval systems, tool calling, observability, evaluation, and expert-level design patterns.

How to use this guide

- Read the roadmap in order if you are new to Spring AI.
- Use each stage as a milestone with mini projects and revision checkpoints.
- Focus on concepts, implementation patterns, and trade-offs rather than memorizing APIs.

Learning progression at a glance

Stage	Focus	Key Topics	Outcome
Basic	Foundations	Java, Spring Boot, REST, prompts, LLM basics, Spring AI abstractions	Build simple chat apps
Intermediate	Application architecture	Memory, structured output, embeddings, vector stores, advisors, testing	Build useful AI features
Advanced	RAG and tooling	Retrieval pipelines, function calling, multimodal flows, guardrails, observability	Ship production-ready services
Expert	Production systems	Evaluation, scalability, reliability, agent orchestration, governance, platform design	Design enterprise AI platforms

Stage 0: Prerequisites before Spring AI

Programming and framework foundations

- Core Java: classes, interfaces, generics, records, streams, exceptions, collections, concurrency basics.
- Spring Boot: dependency injection, configuration properties, profiles, starters, auto-configuration, REST controllers, validation.
- HTTP and JSON: request-response flow, status codes, serialization, authentication headers, timeouts, retries.
- Build tools: Maven or Gradle, dependency management, project structure, test lifecycle, packaging.
- Databases and storage: SQL basics, Redis basics, document storage concepts, vector database fundamentals.

AI and LLM essentials

- What LLMs are, what they are good at, and where they fail.
- Tokens, context window, temperature, top-p, hallucinations, latency, and cost trade-offs.
- Prompting basics: instructions, examples, delimiters, role prompting, output constraints.
- Embeddings and semantic similarity at a conceptual level.
- RAG, tool calling, and memory as separate patterns with different purposes.

Checkpoint project

• Create a standard Spring Boot REST API with validation, externalized configuration, and clean exception handling.

• Then write a plain Java prompt builder and call a model API manually to understand what Spring AI abstracts away.

Stage 1: Spring AI basics

At this stage, the goal is to understand the role Spring AI plays inside a Spring Boot application. You should become comfortable with the abstraction model, configuration flow, and simple text-generation use cases.

Core concepts to learn

- What Spring AI is and why it exists: consistent abstractions across model providers and AI workflows.
- Supported model categories: chat models, embedding models, image models, audio models, moderation models.
- Starter dependencies, provider configuration, API keys, environment variables, profiles, and secrets management.
- ChatClient and model abstraction: sending prompts, receiving responses, handling metadata.
- Prompt composition: system prompts, user prompts, templates, variables, reusable prompt fragments.
- Basic response handling: plain text outputs, role separation, prompt logging considerations.

Basic implementation topics

- Creating a simple AI controller or service in Spring Boot.
- Separating controller, service, prompt template, and provider configuration.
- Handling common errors such as invalid credentials, rate limits, empty responses, and request failures.
- Using configuration properties to switch providers or model names across environments.
- Understanding synchronous flow first before adding streaming or orchestration.

Practice ideas

1. Build a text summarizer endpoint.
2. Build a question-answer endpoint with a strong system prompt.
3. Build a content rewrite service with reusable prompt templates.

Stage 2: Intermediate Spring AI application design

Once the basics are comfortable, move from toy demos to maintainable application design. The focus here is on output control, context handling, modular design, and testability.

Structured outputs and data contracts

- Mapping model output into typed Java objects.
- JSON-oriented prompting, validation, parsing, fallback strategies, and defensive handling of malformed output.
- Separating model creativity from strict application contracts.

Conversation and memory

- Short-term conversation context versus durable application memory.
- When to store messages, when to summarize, and when to discard history.
- Session-scoped context, user-scoped context, and privacy-aware memory design.

Embeddings and vector search basics

- Generating embeddings for documents or chunks.
- Chunking strategy: size, overlap, semantic boundaries, and document normalization.
- Indexing and retrieval flow using a vector store.
- Relevance, top-k, metadata filters, and retrieval quality trade-offs.

Spring integration design

- Service layer design for AI features: orchestration service, retrieval service, prompt service, and output mapper.
- Keeping prompts versioned and testable instead of scattering strings across controllers.
- Using advisors or intercepting layers to inject policies, context, or logging.
- Using Spring profiles to separate local experimentation from production settings.

Intermediate milestone

- Build a support assistant that answers from your own knowledge base.
- Return typed JSON with answer, confidence note, and citation placeholders.
- Store conversation context for one session and add tests for prompt-building logic.

Stage 3: Advanced Spring AI topics

Production-grade RAG

- Document ingestion pipelines: cleaning, chunking, metadata extraction, deduplication, and incremental refresh.
- Retrieval strategies: semantic search, hybrid search, reranking, query rewriting, and contextual compression.
- Grounding prompts with retrieved content while reducing hallucinations and prompt injection risk.
- Designing answer templates that distinguish between retrieved evidence and generated explanation.

Tool calling and workflow execution

- Function calling concepts: when the model should ask for data, trigger actions, or compute through tools.
- Designing safe tool interfaces with clear schemas, validation, authorization, and side-effect control.
- Connecting tools to internal services such as pricing, ticket status, inventory, or knowledge systems.
- Avoiding uncontrolled autonomous behavior by constraining tool scope and execution paths.

Streaming and interactive UX

- Streaming partial responses for chat interfaces or long-running reasoning flows.
- Backpressure, cancellation, timeout boundaries, and user experience trade-offs.
- Designing web and mobile clients that can display incremental tokens safely.

Multimodal and specialized models

- Working with image input, OCR-like tasks, captions, or multimodal prompts where supported.
- Audio or speech workflows at the architecture level: transcription, summary, structured extraction.
- Moderation and safety models for content filtering, abuse control, and policy enforcement.

Reliability and guardrails

- Prompt injection defense patterns for retrieval and tool-calling scenarios.
- Output validation, schema checks, fallback prompts, retries, and human escalation paths.
- Rate limiting, quotas, tenant isolation, request budgets, and graceful degradation.

Stage 4: Expert-level Spring AI engineering

Expert work is less about basic API usage and more about designing AI systems that are observable, evaluable, secure, cost-aware, and evolvable across many teams and use cases.

Architecture and platform thinking

- Designing AI capability as a platform service inside a larger Spring ecosystem.
- Provider abstraction strategy: swapping or routing across providers without breaking business flows.
- Policy-driven architecture for prompts, models, tools, memory, and retrieval.
- Multi-tenant architecture, per-customer isolation, request tracing, and quota governance.

Observability and evaluation

- Capturing prompt, response, latency, token usage, failures, and retrieval metrics.
- Tracing across controller, service, retrieval, model call, tool execution, and persistence layers.
- Offline evaluation sets for answer quality, grounding, tool correctness, and regression detection.
- Online feedback loops, annotation workflows, and continuous prompt-model tuning.

Testing strategy

- Unit tests for prompt builders, chunkers, mappers, validators, and tool adapters.
- Integration tests with mocked model responses and deterministic fixtures.
- Evaluation-driven tests for RAG relevance, answer correctness, and structured output stability.
- Chaos thinking: simulate timeouts, model drift, partial retrieval failure, and malformed outputs.

Security, compliance, and governance

- Secrets handling, encryption, PII minimization, audit logging, and retention policy design.
- Content filtering, abuse controls, approval flows, and organization-specific acceptable-use policy enforcement.
- Safe handling of customer documents, embeddings, chat history, and derived data products.
- Model risk management: legal, privacy, explainability, and operational accountability.

Performance and cost engineering

- Model selection per task instead of one-model-for-everything architecture.
- Caching layers for prompts, retrieval results, embeddings, and deterministic tool data where valid.
- Context compression, summarization, prompt trimming, and retrieval optimization to reduce token waste.
- Latency budgets, asynchronous workloads, batch jobs, and queue-backed orchestration for heavy pipelines.

Topic map: what to study in each level

Stage	Focus	Key Topics	Outcome
Basic	Prompting	Prompt templates, variables, role messages, response basics	Reliable simple prompting
Basic	Spring setup	Starters, configs, beans,	Runnable local project

		provider setup, secret handling	
Intermediate	Output control	Structured responses, validation, parsers, schema discipline	Typed application data
Intermediate	RAG intro	Embeddings, chunking, vector store, metadata, top-k retrieval	Knowledge-grounded answers
Advanced	Agents and tools	Function calling, tool safety, orchestration patterns, loops	Task-capable assistants
Advanced	Operations	Streaming, observability, fallback, rate limiting, guardrails	Production readiness
Expert	Governance	Evaluation, compliance, cost control, platform design, multi-tenancy	Enterprise maturity

Recommended project ladder

Project 1: AI text utility service — Summarization, rewriting, extraction, and classification endpoints with clean service layering.

Project 2: Internal document Q&A — A basic RAG application with ingestion, embeddings, retrieval, and answer generation.

Project 3: Support copilot — Session memory, citations, escalation path, feedback capture, and analytics.

Project 4: Tool-enabled business assistant — Model chooses between retrieval and approved internal tools such as order lookup or ticket status.

Project 5: Production AI platform slice — Tenant-aware architecture, evaluation dashboard concepts, tracing, policies, and governance hooks.

Common mistakes to avoid

- Jumping into RAG before understanding prompting, context windows, and response handling.
- Treating model output as fully trustworthy without validation or fallback logic.
- Storing too much conversation history and creating noisy or privacy-risky prompts.
- Using one prompt for every scenario instead of separating use cases and constraints.
- Ignoring cost, latency, and observability until after the feature ships.
- Letting tool execution become too broad or autonomous without business guardrails.

Suggested study sequence

4. Master Spring Boot fundamentals and externalized configuration.
5. Learn LLM basics, prompting, tokens, and model limitations.

6. Build simple Spring AI text applications.
7. Add structured outputs and typed contracts.
8. Learn embeddings, chunking, vector stores, and RAG foundations.
9. Add session memory, advisors, and test strategy.
10. Build tool-calling and workflow-driven assistants.
11. Improve production readiness with observability, security, rate limits, and fallback paths.
12. Move into evaluation, governance, cost optimization, and platform architecture.

Final outcome

- By the end of this roadmap, you should be able to design far more than a chatbot.
- You should be able to build Spring-based AI systems that retrieve data, call tools, enforce policies, expose typed results, and operate reliably in production.

Prepared as a structured Spring AI study roadmap for progressive learning.